NETWORK FINGERPRINT GENERATOR

Web based Project

SELF SUPPORT
HIRA ISHTIAQ
BSEF24A017

Contents

Proposal Approval Signatures (FAC)

I, the undersigned course instructor, have reviewed the submitted semester project proposal. Based on my assessment, I acknowledge the group's submission and confirm that the proposal has been evaluated for completeness and alignment with the course objectives.

Project Title: Network Fingerprint Tool

Instructor's Name: Ma'am Mufassira

Signature: _____

Date: _____

Chapter 1: Introduction

1.1 Abstract

Understanding network traffic is challenging for networking students because raw packet data is overwhelming and unstructured. Traditional teaching methods rely on command-line tools like tcpdump or Wireshark, which present a high learning curve and lack high-level behavioral summaries.

This project develops a web-based **Network Fingerprint Tool** that captures live network traffic when a user visits a website, analyzes captured packets, and produces a unique *behavioral fingerprint*. The fingerprint summarizes which protocols the site uses, packet size statistics, data transmission frequency, and external servers contacted. Students can compare fingerprints of two websites side-by-side to observe real-world differences — for example, a video-streaming site versus a static news page. Visualizations include protocol distribution pie charts, packet size histograms, and traffic-over-time line graphs.

1.2 Problem Statement

Undergraduate networking students struggle to interpret live network traffic because raw packet dumps contain hundreds of packets with no immediate behavioral insight. Static diagrams and theory alone fail to show how different websites (streaming, social media, API-heavy) exhibit distinct traffic patterns.

This project builds a web-based tool that:

- Captures live traffic using Scapy while accessing a target URL
- Extracts meaningful features (packet sizes, protocols, inter-arrival times, unique IPs)
- Generates a structured JSON fingerprint summarizing site behavior
- Classifies website behavior as Streaming, Social Media, Static Content, or API-heavy
- Enables side-by-side comparison of two websites with visual difference indicators

1.3 Objectives Achieved

- Develop a packet capture module that sniffs network traffic only from the entered URL (avoiding background noise)
- Extract key features: packet size statistics (mean, min, max), protocol distribution (TCP, UDP, DNS, HTTPS), total bytes transferred, unique destination IPs, and DNS query names
- Generate a structured network fingerprint JSON containing all aggregated statistics
- Implement a rule-based classifier that labels website behavior into four categories with confidence percentages
- Create a side-by-side comparison view highlighting differences in total bytes, unique IPs, mean packet size, and behavior labels
- Visualize traffic characteristics using Chart.js (protocol pie chart, packet size histogram, traffic timeline)

Chapter 2: Development Methodology

The development of the Network Fingerprint Tool followed the **Agile (Scrum)** methodology. Agile was selected due to the iterative nature of network processing — packet capture logic, feature extraction accuracy, and real-time visualization required frequent adjustments based on testing with different websites (e.g., YouTube vs. a plain HTML page). The team worked in two-week sprints, delivering successively working modules: capture, extraction, fingerprinting, classification, and finally frontend integration.

Project Timeline

Day	Duration	Deliverable
Day 1	Day 1	Set up Flask backend + Scapy packet capture with URL filtering
Day 2	Day 2	Feature extraction module (extract.py) — packet sizes, protocols, unique IPs, total bytes
Day 3	Day 3	Fingerprint JSON generation + rule-based classifier (classify.py)
Day 4	Day 4	Flask API endpoints (/api/analyze, /api/compare) + basic frontend HTML/CSS
Day 5	Day 5	JavaScript integration, Chart.js visualizations (pie chart, histogram, timeline), side-by-side comparison view, and end-to-end testing

Chapter 3: Key Functionalities

The project successfully delivered a fully functional Network Fingerprint Tool with the following features:

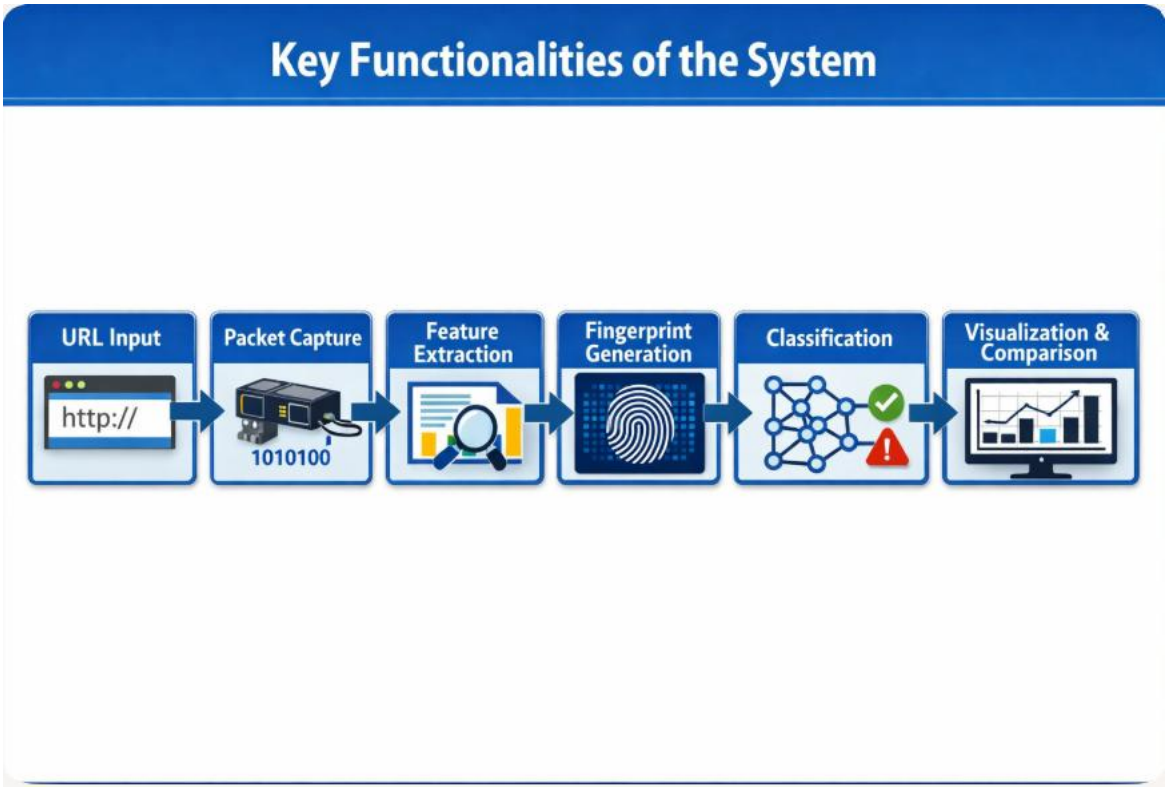

Figure 1: Key Functionalities of the System

Chapter 4: System Design

4.1 Layers of System

The Network Fingerprint Tool is structured into three distinct layers:

4.1.1 Input Layer

- User enters a target URL (e.g., https://example.com) in a text field
- JavaScript validates URL format (must start with http:// or https://) before submission
- Optional second URL field enables comparison mode

- Input is sent via POST request to Flask backend

4.1.2 Capture & Analysis Layer (Backend)

- Flask receives URL and triggers Scapy packet sniffer on active network interface
- Capture runs for configurable duration (default: 10 seconds)
- Simultaneously, an HTTP/HTTPS request is made to the target URL to generate real traffic
- After capture, extract.py reads the .pcap file and computes: total packet count, protocol breakdown percentages, packet size statistics (mean/min/max), unique destination IPs, DNS query names, total bytes transferred, and inter-arrival times
- fingerprint.py assembles all statistics into a structured JSON object
- classify.py applies rule-based logic to assign behavior label (Streaming / Social Media / Static Content / API-heavy / Unknown)

4.1.3 Output Layer (Frontend Visualization)

- JavaScript receives JSON fingerprint from Flask API
- Summary card displays: site URL, timestamp, total packets, total bytes, top protocol, unique IPs, DNS queries, mean packet size, behavior label
- Chart.js renders three visualizations:
    - Protocol distribution pie chart
    - Packet size histogram (bar chart)
    - Traffic timeline (line chart of packets per second)
- Comparison mode renders two fingerprint cards side-by-side with color-coded difference indicators (green = lower, red = higher)

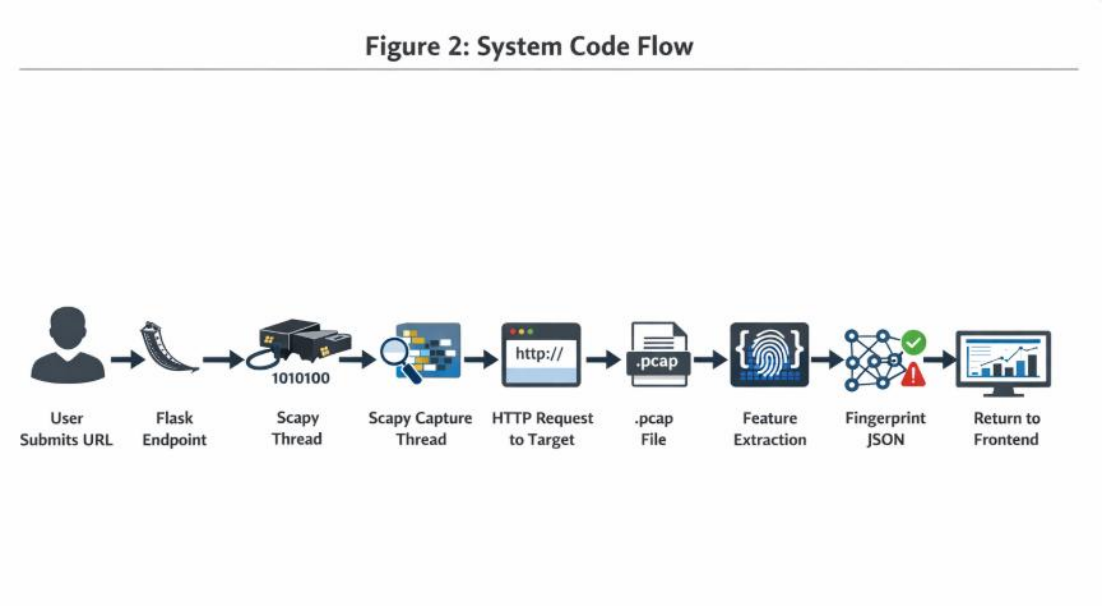

Figure 2: System Code Flow

Chapter 5: Technology Stack

Category	**Technology**
Backend Language	Python 3.x
Packet Capture	Scapy
Web Framework	Flask
Frontend Languages	HTML, CSS, JavaScript
Charting Library	Chart.js
Data Formats	pcap (raw packets), JSON (fingerprint exchange)
Version Control	Git & GitHub

Chapter 6: Comparative Analysis of Features

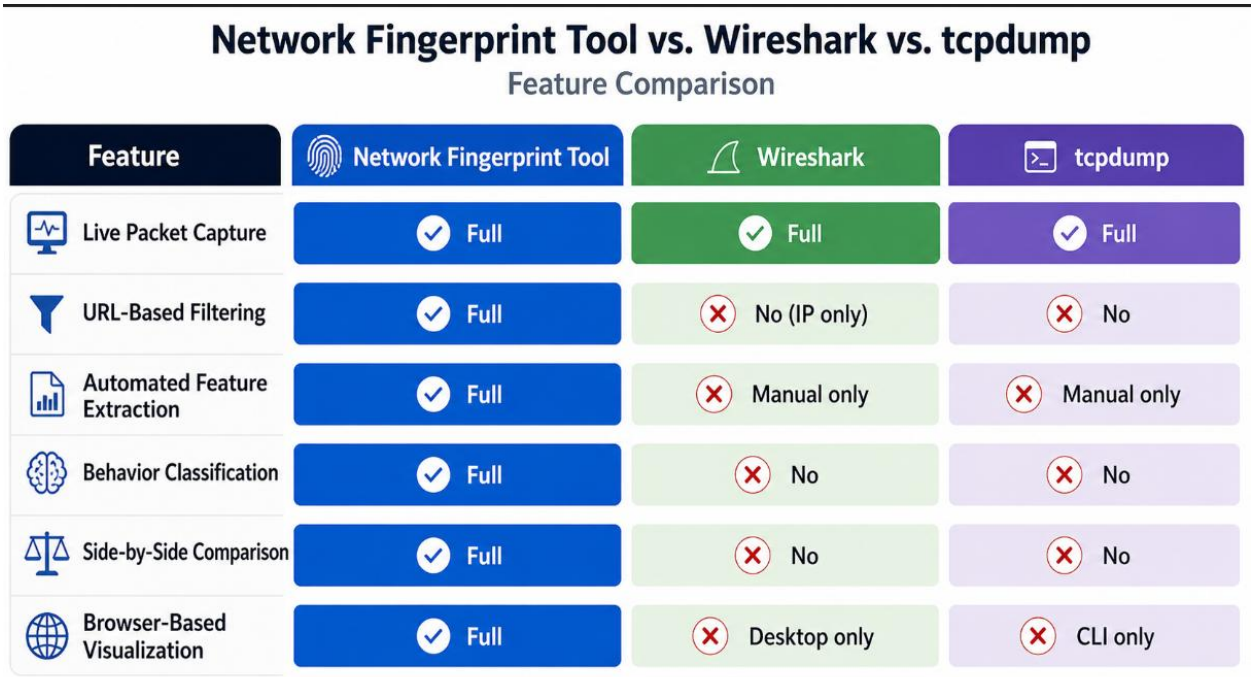

Figure 3: Feature Comparison Chart

Feature	**Network Fingerprint Tool**	**Wireshark**	**tcpdump**
Live Packet Capture	✓ Full	✓ Full	✓ Full
URL-Based Filtering	✓ Full	✗ No (IP only)	✗ No
Automated Feature Extraction	✓ Full	✗ Manual only	✗ Manual only
Behavior Classification	✓ Full	✗ No	✗ No

Feature	Network Fingerprint Tool	Wireshark	tcpdump
Side-by-Side Comparison	✓ Full	✗ No	✗ No
Browser-Based Visualization	✓ Full	✗ Desktop only	✗ CLI only

Chapter 7: System Use Cases

UC-01: Analyze Single Website Fingerprint

Field	Description
Use Case ID	UC-01
Use Case Name	Analyze Single Website Fingerprint
Primary Actor	Networking Student
Trigger	User wants to see behavioral fingerprint of a website
Pre-Condition	Tool is open in browser; network interface is active
Post-Condition	Fingerprint summary card and three visualizations are displayed

Field	Description
Success Scenario	1. User enters a valid URL (e.g., https://youtube.com) 2. User clicks "Analyze" button 3. System captures packets for 10 seconds while accessing the URL 4. System extracts features and generates fingerprint JSON 5. Classifier assigns behavior label (e.g., "Streaming") 6. Frontend displays summary card, protocol pie chart, packet size histogram, and traffic timeline
Abort Scenario	1. User enters invalid URL format → inline error message shown 2. Network interface unavailable → error: "Could not reach the website, check your URL" 3. No packets captured → error: "No traffic detected for this URL"

UC-02: Compare Two Websites Side-by-Side

Field	Description
Use Case ID	UC-02
Use Case Name	Compare Two Websites Side-by-Side
Primary Actor	Networking Student
Trigger	User wants to contrast network behavior of two different sites
Pre-Condition	Tool is open; both URLs are valid

Field	Description
Post-Condition	Two fingerprint cards displayed with difference highlights
Success Scenario	1. User enters first URL (e.g., https://youtube.com) 2. User enters second URL (e.g., https://example.com) 3. User clicks "Compare" button 4. System captures traffic for both URLs sequentially 5. System generates two fingerprints and a diff object 6. Frontend displays two cards side-by-side 7. Differences are color-coded: green for lower value, red for higher value 8. Comparison shows which site used more bytes, more IPs, larger packets, and behavior labels
Abort Scenario	One URL fails to load → System displays fingerprint for successful site only and error for failed site

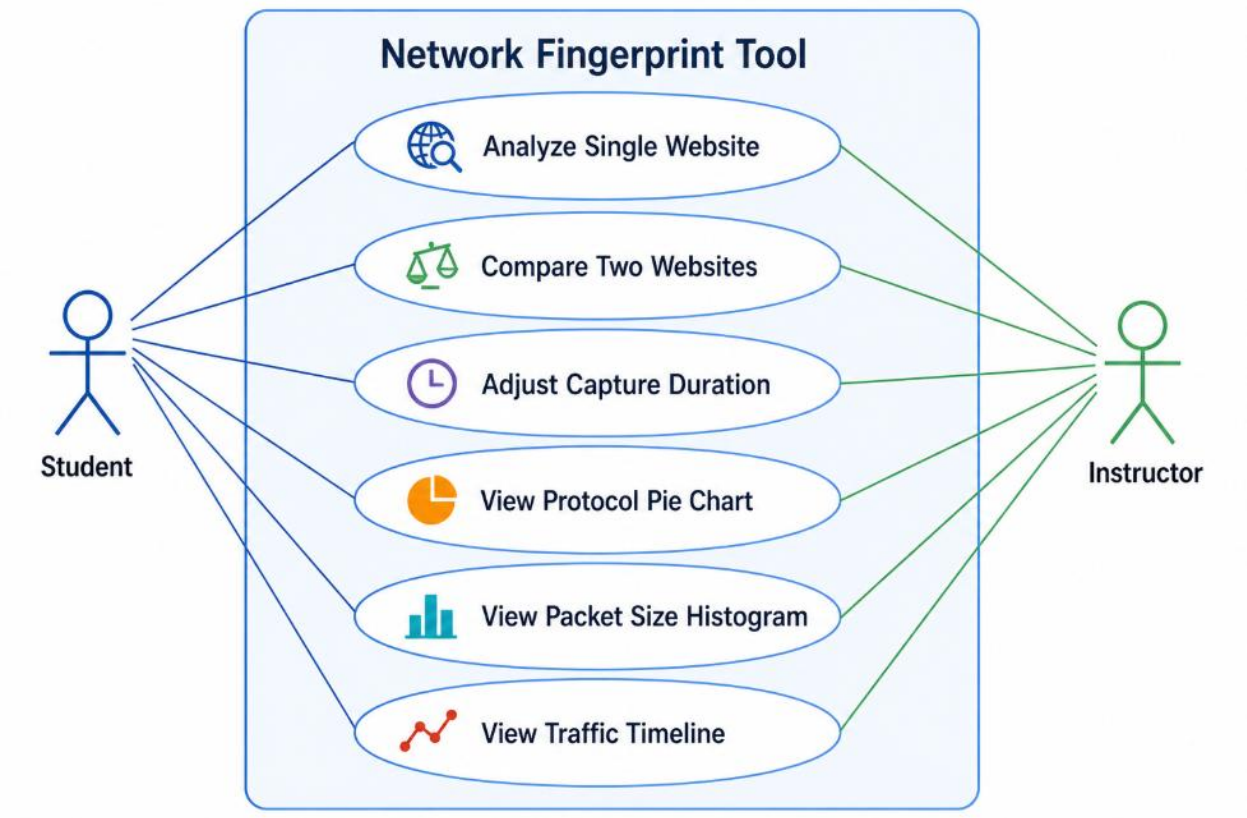

Figure 4: Use Case Diagram

Chapter 8: Test Cases

Test Case ID	Test Case Description	Pre-Condition	Test Steps	Expected Result
TC-01	Verify URL validation rejects invalid formats	Frontend loaded	1. Enter "example" (no protocol) 2. Click Analyze	Inline error message appears; no server request sent

Test Case ID	Test Case Description	Pre-Condition	Test Steps	Expected Result
TC-02	Verify valid URL is accepted	Frontend loaded	1. Enter "https://example.com" 2. Click Analyze	POST request sent to /api/analyze with URL payload
TC-03	Verify packet capture captures only target URL traffic	Network active	1. Enter "https://example.com" 2. Capture runs 10 seconds	Packets not from example.com do main are filtered out
TC-04	Verify feature extraction produces all required statistics	.pcap file exists	1. Run extract.py on captured pcap	Dictionary contains: total_packets, protocol_breakdown, packet_sizes (list), unique_ips, total_bytes, inter_arrival_times
TC-05	Verify classifier labels streaming site correctly	Capture from video site (e.g., YouTube)	1. Analyze https://youtube.com	Behavior label = "Streaming" with confidence >80%

Test Case ID	Test Case Description	Pre-Condition	Test Steps	Expected Result
TC-06	Verify classifier labels static site correctly	Capture from simple HTML page	1. Analyze http://example.com	Behavior label = "Static Content" or "Unknown"
TC-07	Verify side-by-side comparison shows diff highlights	Two valid fingerprints exist	1. Enter URL A and URL B 2. Click Compare	Both cards shown; differences highlighted red/green
TC-08	Verify protocol pie chart renders correctly	Fingerprint JSON received	1. Analyze any URL	Chart.js pie chart displays TCP, UDP, DNS percentages
TC-09	Verify packet size histogram renders	Fingerprint JSON received	1. Analyze any URL	Bar chart groups packet sizes (e.g., 0-100B, 100-500B, 500-1500B)
TC-10	Verify traffic timeline renders	Fingerprint JSON received	1. Analyze any URL	Line chart shows packet count per second over capture window

Chapter 9: Project Repository

GitHub Repository:

HiraIshtiaq/Network_Fingerprint_Generator

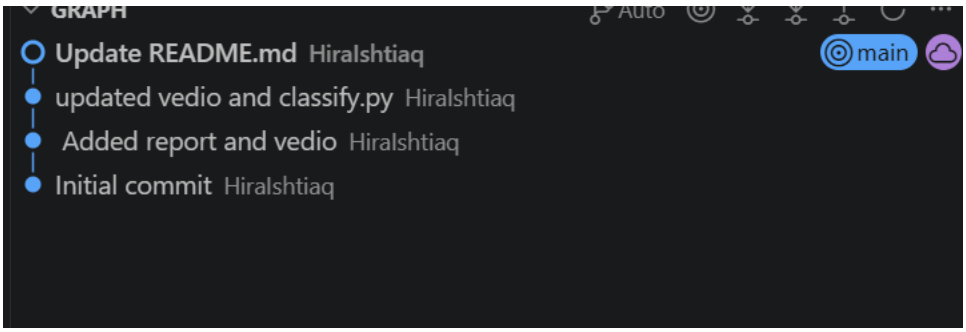

Figure 6: GitHub Commit History

Chapter 10: Conclusion

The Network Fingerprint Tool successfully achieved all stated objectives. The platform captures live network traffic when a user visits a website, analyzes captured packets, and produces a unique behavioral fingerprint summarizing protocol usage, packet size statistics, data frequency, and contacted servers.